

突击班-并发编程（第一天）

一、JDK21的特性，虚拟线程，虚拟线程与普通线程的区别？（通力电梯）

首先内存占用更小，而且创建和销毁的成本更低，并且不存在上下文切换以及阻塞、唤醒带来的时间成本，甚至在JDK21种提供的构建虚拟线程的方式，都没提供管理数量的API。

底层实现：

- 普通线程，依然是依赖于操作系统的。
- 虚拟线程，由JVM调度的。

创建、销毁成本：

- 普通线程的创建成本相对会比较高，因为依赖OS构建。创建成本相对比较高。
- 虚拟线程直接依赖JVM构建。速度特别快，成本也低。

内存、数量成本：

- 普通线程，基本需要1M内存左右，一般情况下，普通线程数量上千就会因为频繁的上下文切换，导致处理速度比较慢。
- 虚拟线程，占用内存特别小，而且因为不涉及上下文切换的情况，数量可以达到上万，甚至十万，百万都是ok的。

阻塞、唤醒成本：

- 普通线程因为在阻塞跟唤醒时，会涉及到用户态与内核态的切换，导致阻塞、唤醒的时间成本比较高。
- 虚拟线程因为不受OS管，So，他的阻塞、唤醒时间成本极低。

Ps：虚拟线程的出现是为了解决IO密集的任务，而CPU密集的任务虚拟线程也没招。

二、ConcurrentHashMap在1.7和1.8区别。（58）

** 这两区别，主要聊结构跟锁机制的区别，扩容以及size方法，可聊可不聊。 **

结构：

- JDK1.7：数组 + 链表
- JDK1.8：数组 + 链表 + 红黑树

锁机制：

- JDK1.7：Segment (ReentrantLock)，分段锁的概念，默认16个Segment，每个Segment下都有对应的Entry (key-value)。
- JDK1.8：CAS + synchronized，粒度相对JDK1.7更细，每个桶的位置都是独立的一把锁。如果数据要扔到数组上，采用CAS，挂到链表、红黑树上，采用synchronized。

扩容方式：

- JDK1.7：需要全局加锁，锁定所有的Segment，然后再做初始化新数组、迁移数据，rehash等操作。

- JDK1.8: 依然会基于一个标记去锁定, 但是他不会阻塞查询操作, 会“阻塞”写操作。如果些操作发现正在扩容, 会去协助扩容 (迁移数据), 等扩容完毕, 数据就可以正常写入了。

size方法:

- JDK1.7: 会累加所有Segment下对应的count, 如果在累加过程中有并发写操作, 则会锁定所有的Segment然后再重新累加。
- JDK1.8: 因为JDK1.8里面采用的是LongAdder的机制, 直接把LongAdder里所有的CounterCell, 以及CHM本身的baseCount直接累加返回, 不考虑并发写的情况。

三、ConcurrentHashMap的线程安全机制? (税友)

(只要没说版本, 咱就聊JDK1.8+)

如果能把控住, 最好都聊, 如果感觉把控不好, 第一个为核心。

1、在写入数据时, 会基于CAS + synchronized确保写入时的原子性。

- 在写入数据到数组时, 会基于CAS将数据写入。
- 在写入数据到链表、红黑树时, 会基于synchronized锁住当前桶位置

2、初始化数组时的安全保证

- 因为CHM默认 数组 在当前对象中有一个即可, 而且CHM对于 数组 的构建是懒加载的。
- 为了确保第一次构建时的线程安全, 基于一个属性sizeCtl来控制的。哪个线程优先将sizeCtl属性基于 **CAS** 修改为-1, 哪个线程就去构建数组。

3、计数器的线程安全

- CHM中写入一个数据, 计数器+1, 删除一个数据, 计数器-1。
- CHM用的是LongAdder, 为何没用Atomic原子类。
 - Atomic原子类底层是基于do-while循环去执行CAS, 直到成功为止。
 - LongAdder类似分段的效果, 不限一个属性去CAS, 提供了多个属性可以去做±1的操作。

4、写入数据到红黑树、查询数据到红黑树的并发情况。

- 红黑树在被写入时, 为了保证线程安全, 需要旋转变色, 如果涉及到了左旋、右旋、指针会变化, 如果有并发的查询操作在这个红黑树里, 可能会出现查询不到本来就存在数据的情况。
- CHM依然是基于CAS维护了一个属性lockState, 如果有写操作, 不会让读线程来红黑树查询, 让读线程去查询保留的双向链表。同理有读线程在查询红黑树时, 写操作会阻塞。

四、ConcurrentHashMap的协助扩容是什么? (中软外包)

优先了解一下扩容的过程, 核心的两步:

- 创建新数组 (一个线程来搞就可以, 不涉及协助情况, 最先出发扩容的线程创建新数组)
- 将老数组的数据迁移到新数组 (可以来协助将数据做迁移)

多个线程去线程, 可能在线程安全的问题, CHM解决的方式是, 先领任务 (CAS), 再做迁移。每个线程负责的索引位置不同, 不会出现并发安全问题。

线程迁移 可能放到新数组 可能放到老数组 ，咋控制的了？（同学张）

线程在协助扩容时，可能是将数据直接放到老数组，不做协助扩容，还有可能是先去协助迁移数据，扩容完毕后，将数据扔到新数组。

写数据的线程什么时候去协助扩容：

- 如果写线程要写的数据需要放的索引位置已经被迁移完毕并且填充上的ForwardingNode（hash=-1），那就直接去协助扩容，完成扩容后，再将数据写到新数组。
- 如果写线程要写的索引位置，没有被填充ForwardingNode，那就正常完成写入的操作。

五、ConcurrentHashMap的get会阻塞嘛？（同程）

get操作不会发生阻塞的情况。特殊情况就下面这两个。

红黑树的点：

- 因为红黑树在被写时，会发生左旋右旋的情况，读线程不能直接查询，读线程会去查询保留的双向链表，不会发生阻塞。

扩容的点：

- 在扩容时，老数组的数据可能已经被迁移到新数组了，此时无法查询到具体的数据。因为扩容迁移完毕的位置会放上ForwardingNode，他有一个标记是hash = -1。看到这个标记直接去新数组位置查询具体的数据。

六、ThreadLocal了解过嘛？（宇信科技）（光大银行）

ThreadLocal最常见的操作就是 **当前线程传参**。

更多的聊一下原理。

ThreadLocal本质是Thread对象中的一个Map里得key。

存储数据的是Thread中的Map，而ThreadLocal作为key，value是咱们自己要存储的数据。

每次基于ThreadLocal去执行set以及get时，本质就是

- set：将ThreadLocal作为key，写入的参数作为value，扔到Thread中的Map里。
- get：将ThreadLocal作为key，查询Thread中的Map对应的value。

父子传参：

- 当前线程创建了Thread对象t1，那么此时父线程是当前线程，子线程就是t1。在这个情况下，就可以采用InheritableThreadLocal实现父子线程之间的传参。
- 本质其实就是在 创建子线程 时，会将父线程的InheritableThreadLocal中的数据扔到子线程的InheritableThreadLocal中。下面这段代码就是new Thread()子线程时的赋值位置。

```
if (inheritThreadLocals && parent.inheritableThreadLocals != null)
    this.inheritableThreadLocals =
        ThreadLocal.createInheritedMap(parent.inheritableThreadLocals);
```

线程池传参：

- 在使用线程池时，线程池里得线程基本都是复用的，投递任务的线程跟线程池里得工作线程几乎没有父子关系了，那就不能采用InheritableThreadLocal去做传参了。
- JUC包下，没有提供线程池传参得ThreadLocal，需要使用三方的：

- 阿里提供的：TransmittableThreadLocal，他的原理是包装任务，将任务Runnable进行一次封装，在创建任务时，将提交任务的线程的ttl中的数据放到任务里的属性中，然后再次仍到这个工作线程的ttl里。这样就可以在执行任务时，从ttl中获取提交任务的线程中的数据了。
- 开源的DynamicTP：在动态线程池里也提供了任务的增强，方式跟阿里的TransmittableThreadLocal几乎一致。

七、synchronized 和ReentrantLock是乐观锁还是悲观锁？悲观锁是什么？乐观锁什么？（中石油）

synchronized 和ReentrantLock在Java中的定位是悲观锁。

Java中的乐观锁实现是CAS（Compare and Swap）。

乐观锁：认为，没有线程跟我有并发操作，直接操作，操作完确认下有并发没。

悲观锁：认为，必然有现成跟我有并发操作，先获取到对应的资源后，再去执行具体操作。

在Java中有个怪怪的点，CAS没啥说的，就是直接比较和交换，成功后，返回true代表没并发。如果有并发，会返回false。

不过Java中的悲观锁的底层实现，都是基于CAS去完成的。synchronized无论是轻量级锁还是重量级锁底层都会基于CAS去修改synchronized中的owner的属性。ReentrantLock底层也是基于CAS去修改state属性。

当然也可以扩展synchronized的底层，锁升级之类的。也可以扩展到ReentrantLock的底层AQS相关的内容。

八、CountDownLatch有了解过嘛？（阿里乌鸦）

CountDownLatch本身是JUC包下的一个工具，他的功能就是计数器。

他不是往上+，而是往下-。一般是配合线程池，去并行处理任务时用的比较多。

比如现在要基于线程池去并行处理三个任务，等到这三个任务都执行完毕，再由业务线程去执行后续业务代码时，就可以采用CountDownLatch

```
CountDownLatch latch = CountDownLatch(3);  
并行处理任务1 --- latch.countDown(); // 任务执行完毕，对CountDownLatch -1  
并行处理任务2 --- latch.countDown(); // 任务执行完毕，对CountDownLatch -1  
并行处理任务3 --- latch.countDown(); // 任务执行完毕，对CountDownLatch -1  
latch.await(); // 业务线程等待上面的三个任务完成，CountDownLatch 计数器为0，唤醒业务  
线程继续向下执行。  
// 业务代码
```

实现原理是基于AQS实现的，本质最核心的是用到了他的一个state属性。是一个int类型。

初始化 CountDownLatch 时，设置上具体的数值，每次countDown，对state - 1，等到state减到0，就会唤醒await的线程。。